

The background features abstract, overlapping green geometric shapes in various shades of green, primarily concentrated on the right side of the slide. The text is positioned on the left side of the slide.

ARM REVIEW 4

TEAM 16

Team Members:

Roll Number

USN

Bhoomi B Hallur

504

01fe23bec235

Prachi Khatawate

511

01fe23bec244

Sagargouda Patil

514

01fe23bec247

Sakshi Mane

566

01fe23bec218

PROBLEM STATEMENT

Build an application to demonstrate the use of I2C protocol and ADC with necessary assumptions. Refer data sheet and manual for implementation.

Real World Application : Humidity sensors play a crucial role in the food dehydration process by continuously monitoring the moisture level inside drying chambers. They help regulate temperature and airflow to ensure uniform drying, prevent microbial growth, and maintain product quality. By providing real-time feedback, these sensors enable efficient energy use and automate drying cycles, making them essential in both small-scale and industrial food processing units.

HUMIDITY SENSORS

Humidity sensors are widely used electronic devices designed to detect and measure the presence of moisture in the air, commonly expressed as relative humidity (RH). These sensors operate on various principles, including capacitive, resistive, and thermal conductivity. As the surrounding humidity changes, the sensor's electrical characteristics change accordingly, making them ideal for applications in weather monitoring, HVAC systems, agriculture, and industrial automation.

Key Characteristics and Applications

Key Features

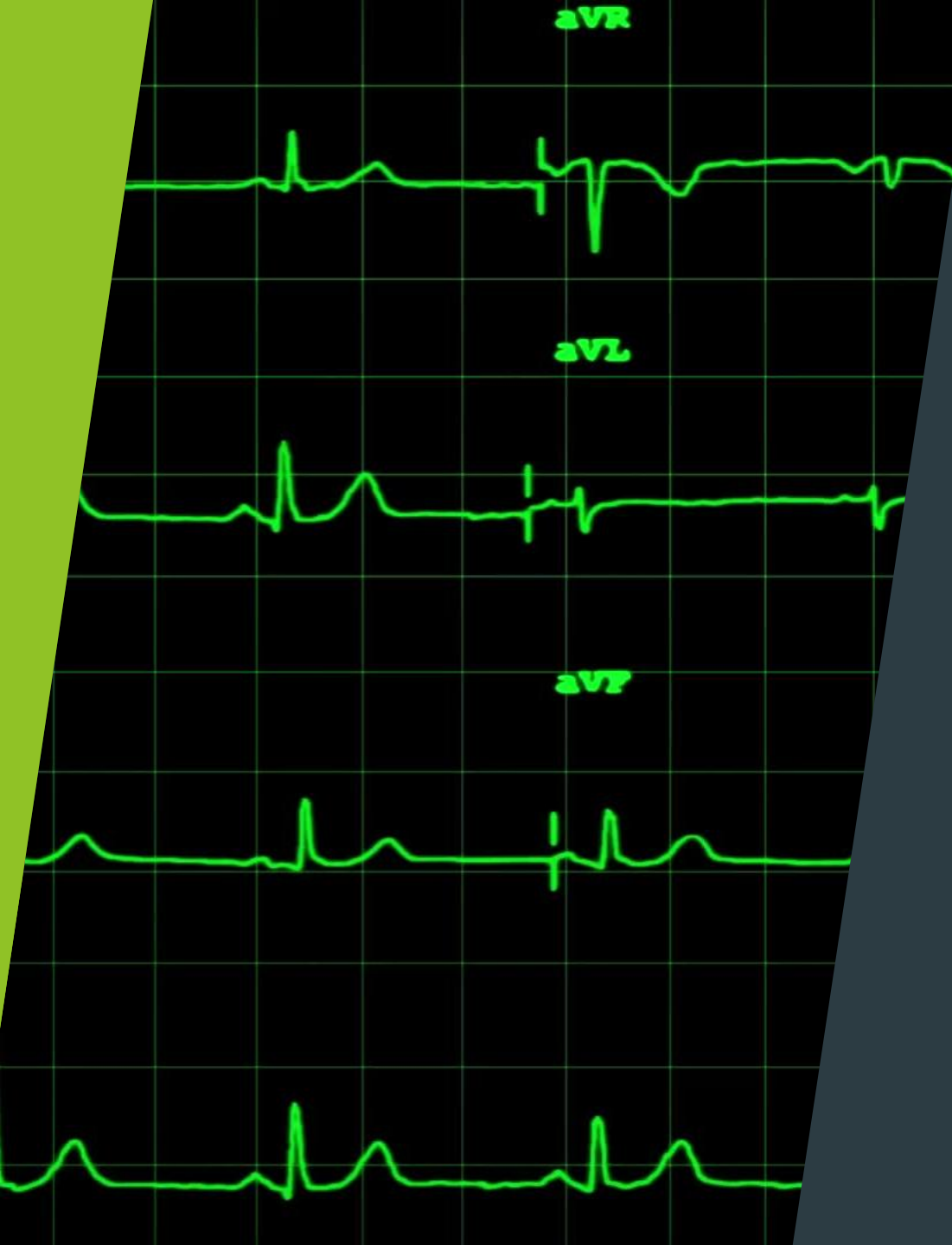
- **Good Sensitivity:** Effective detection of a wide range of humidity levels.
- **High Accuracy:** Especially accurate in measuring relative humidity in both indoor and outdoor environments.
- **Long Life and Low Cost:** Durable and economically viable for continuous humidity monitoring.
- **Simple Interface Circuitry:** Easy to integrate into embedded and microcontroller-based systems.

Applications

- Environmental Monitoring Systems: Used for tracking humidity levels in meteorological and agricultural stations.
- HVAC Systems: Incorporated into heating, ventilation, and air conditioning systems to regulate air quality.
- Industrial Automation: Applied in processes where moisture control is critical.
- Consumer Electronics: Integrated into smart devices for indoor comfort and air quality monitoring.

Technical Specifications

- Model: HS1101 (example)
- Sensor Type: Capacitive or Resistive
- Encapsulation: Plastic housing or epoxy coating
- Detection Parameter: Relative Humidity (%RH)
- Humidity Range: Typically 0%-100% RH
- Operating Voltage (Vcc): 3.3V to 5V DC
- Load Resistance (RL): Defined by circuit design, usually in the range of 10k Ω
- Response Time: Typically < 10 seconds for 63% of full step change
- Sensitivity: $\pm 2\%$ RH or better
- Operating Conditions: 0°C to 60°C; 20%-90% RH (non-condensing)
- Stability: Long-term drift typically < $\pm 1\%$ RH/year

The image shows four ECG waveforms stacked vertically on a black grid background. The top waveform is labeled 'aVR' in red, the second 'aVL' in red, and the third 'aVF' in red. The bottom waveform is unlabeled. Each waveform shows a series of cardiac cycles with distinct P, Q, R, and S waves. The background of the slide features abstract green and blue geometric shapes.

ADC CALCULATION

- ▶ Given supply voltage is 5v
- ▶ So the moisture detects exactly at 2.5 i.e, half of 5v, as our LIC project has a voltage divider which exactly gives this value.
- ▶ IF we need ADC value then
- ▶ $(2.5/5) \times 1023 = 510$

CODE

```
#include <lpc214x.h>
#include <stdio.h>
#define LCD_ADDR 0x27 // Change to 0x3F if LCD doesn't respond
char display_str[32];
// Simple delay function
void delay(unsigned int time)
{
    unsigned int i, j;
    for (i = 0; i < time; i++)
        for (j = 0; j < 800; j++);
}
// Initialize I2C on P0.27 (SDA), P0.28 (SCL)
void init_i2c()
{
    PINSEL0 |= 0x00000050; // Select SDA/SCL function
    I2C0CONCLR = 0x6C; // Clear control bits
    I2C0SCLH = 60;
    I2C0SCLL = 60;
    I2C0CONSET = 0x40; // Enable I2C
}
```

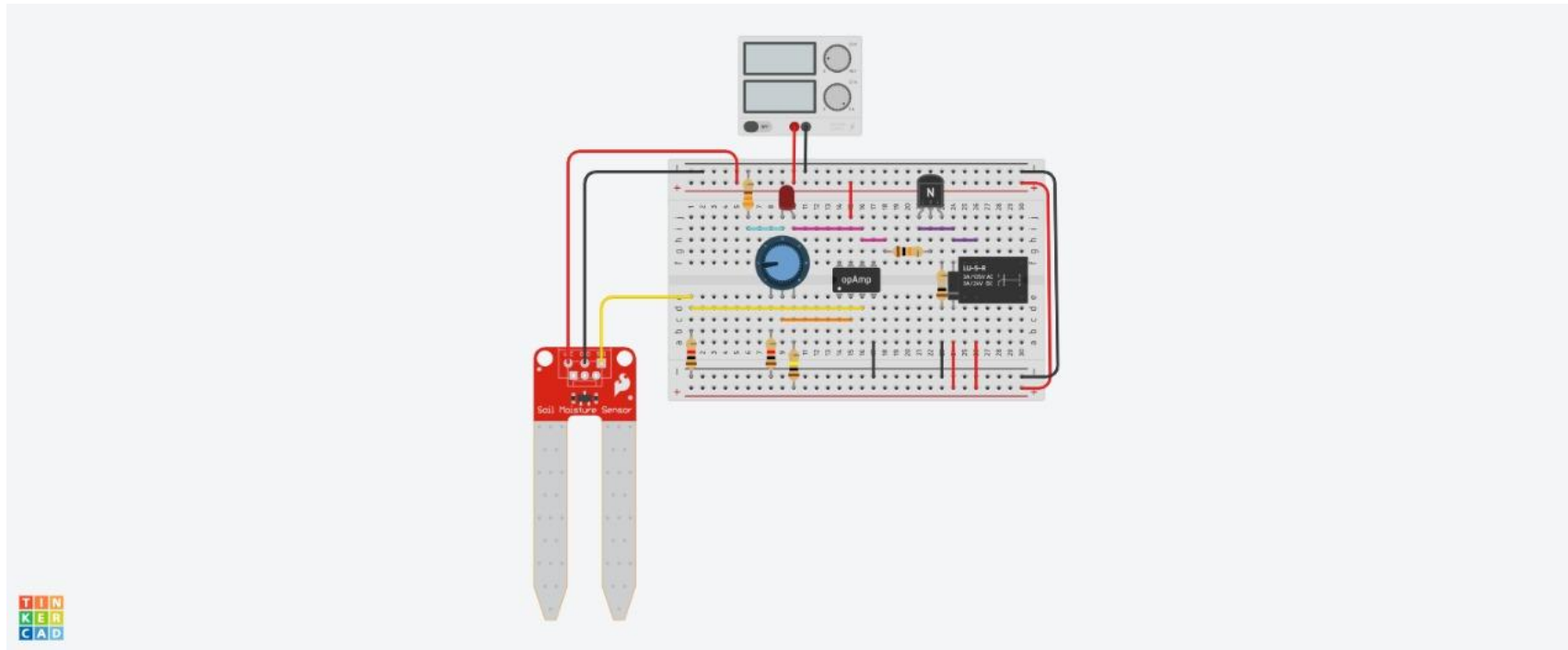
```
void i2c_start()
{
    I2C0CONSET = 0x20;          // STA = 1
    while (!(I2C0CONSET & 0x08)); // Wait for SI
    I2C0CONCLR = 0x28;          // Clear STA and SI
}
void i2c_stop()
{
    I2C0CONSET = 0x10; // STO = 1
    I2C0CONCLR = 0x08; // Clear SI
    delay(5);
}
void i2c_write(unsigned char data)
{
    I2C0DAT = data;
    I2C0CONCLR = 0x08; // Clear SI
    while (!(I2C0CONSET & 0x08)); // Wait for SI
    delay(1); // Critical for stability
}
void lcd_send(unsigned char data, unsigned char is_data)
{
    unsigned char upper = data & 0xF0;
    unsigned char lower = (data << 4) & 0xF0;
    unsigned char mode = is_data ? 0x01 : 0x00;
```

```
unsigned char backlight = 0x08;
i2c_start();
    i2c_write(LCD_ADDR << 1);
    i2c_write(upper | mode | backlight | 0x04);
    i2c_write(upper | mode | backlight);
    i2c_write(lower | mode | backlight | 0x04);
i2c_write(lower | mode | backlight);
    i2c_stop();
}
void init_lcd()
{
    delay(100);
    lcd_send(0x33, 0); // Initialization
    lcd_send(0x32, 0); // 4-bit mode
    lcd_send(0x28, 0); // 2-line, 5x8
    lcd_send(0x0C, 0); // Display ON, Cursor OFF
    lcd_send(0x06, 0); // Auto-increment
    lcd_send(0x01, 0); // Clear display
    delay(50);
}
void lcd_cursor(unsigned char row, unsigned char col)
{
```

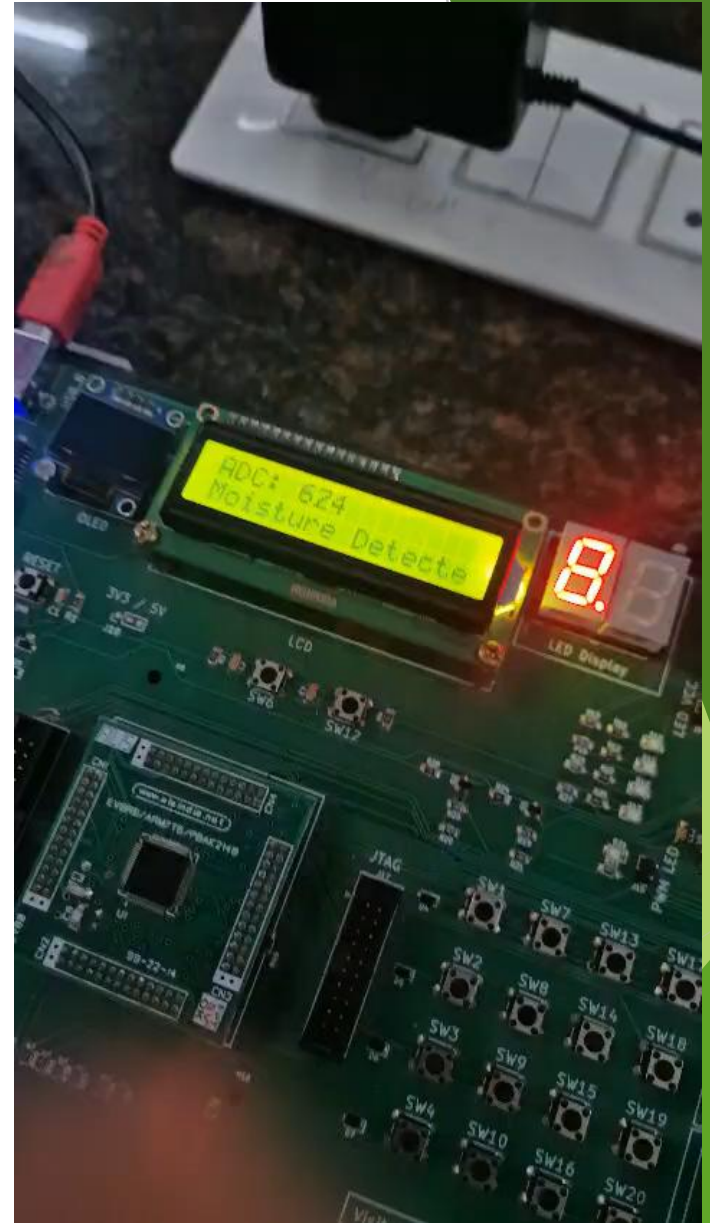
```
unsigned char addr = (row == 0) ? (0x80 + col) : (0xC0 + col);
lcd_send(addr, 0);
}
void lcd_print(char* str)
{
    while (*str)
    {
        lcd_send(*str++, 1);
    }
}
// Configure ADC and read from AD0.4 (P0.25)
unsigned int read_adc()
{
    static int adc_init = 0;
    if (!adc_init)
    {
        PINSEL1 |= (1 << 18); // Enable AD0.4
        adc_init = 1;
    }
    AD0CR = 0x09200410; // SEL=4, CLKDIV=10, PDN=1
    while (!(AD0GDR & (1 << 31))); // Wait for DONE
    return (AD0GDR >> 6) & 0x03FF; // Extract 10-bit result
}
```

```
int main()
{
    unsigned int adc_result;
    init_i2c();
    init_lcd();
    while (1)
    {
        adc_result = read_adc();
        lcd_cursor(0, 0);
        sprintf(display_str, "ADC: %u    ", adc_result);
        lcd_print(display_str);
        lcd_cursor(1, 0);
        lcd_print("                "); // Clear second line
        lcd_cursor(1, 0);
        if (adc_result > 512)
        {
            lcd_print("Moisture Detected");
        }
        else
        {
            lcd_print("Dry (Low Moisture)");
        }
        delay(1000);    }
}
```

PROTEUS SIMULATION



VIDEO OF MOISTURE DETECTION





Moisture
Detected



Low
moisture

Memory usage and execution time details

The screenshot displays the Keil uVision4 IDE interface, showing the project file `C:\Keil\ARM\Examples\Blinky\startup.uvproj` and the `review4.c` source file. The **Registers** window on the left lists the current values of various registers, including `R0` through `R15`, `CPSR`, and `SPSR`. The **Performance Analyzer** window on the right shows the execution time for the `review4.c` module, with a total time of 1.250116 seconds and a time percentage of 2%. The **Command** window at the bottom left displays the memory usage: `*** Currently used: 2008 Bytes (6%)`. The **Call Stack + Locals** window on the bottom right shows the current call stack, with the `main` function at address `0x000003EC` and the `adc_result` variable at address `<not in scope>`. The **ASSIGN** window at the bottom shows the assignment of the `adc_result` variable to the `main` function.

Registers

Register	Value
R0	0x40000088
R1	0x40000088
R2	0x40000088
R3	0x40000088
R4	0x00000000
R5	0x40000024
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x000007D4
R11	0x00000000
R12	0x000003EC
R13 (SP)	0x40000488
R14 (LR)	0x000000FC
R15 (PC)	0x000003EC
CPSR	0x600000D3
SPSR	0x00000000

Performance Analyzer

Module/Function	Calls	Time(Sec)	Time(%)
review4.c	1	1.250116	2%

review4.c

```
90 while (!(AD0GDR & (1 << 31))); // Wait for DONE
91 return (AD0GDR >> 6) & 0x03FF; // Extract 10-bit result
92 }
93
94 int main() {
95     unsigned int adc_result;
96
97     init_i2c();
98     init_lcd();
99
100     while (1) {
101         adc_result = read_adc();
102
103         lcd_cursor(0, 0);
104         sprintf(display_str, "ADC: %u", adc_result);
105         lcd_print(display_str);
106
107         lcd_cursor(1, 0);
108         lcd_print(" "); // Clear second line
109         lcd_cursor(1, 0);
110
111         if (adc_result > 512) {
112             lcd_print("Moisture Detected");
113         } else {
114             lcd_print("Dry (Low Moisture)");
115         }
116     }
117 }
```

Command

```
*** Currently used: 2008 Bytes (6%)
```

Call Stack + Locals

Name	Location/Value	Type
main	0x000003EC	int f()
adc_result	<not in scope>	auto - unsigned int

ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE DEFINE DIR

Real-Time Agent: Target Stopped Simulation t1: 0.00007633 sec L:97 C:1 CAP NUM SCRL OVR RAW

THANK YOU